

Vehicle Detection and Counting Using YOLOv8

Prepared by:

Masoumeh Ostvar

Department of Computer Engineering

Spring 2024

Contents

1. Dataset Preparation	2
2. Model Training	2
3. Vehicle Detection and Counting.....	3
4. Detection Results	6

1. Dataset Preparation

The dataset used in this project was prepared and annotated using Roboflow. The dataset was exported in the YOLOv8 format, which automatically generated the required configuration file (dataset.yaml).

The dataset consists of three object classes:

- Bus
- Car
- Truck

The dataset.yaml file specifies the locations of the training, validation, and testing images and defines the class labels used during training.

```
C: > Users > Mina > Desktop > traffic-monitoring-system-yolov8 > traffic-monitoring-system-yolov8 > Detection > dataset > ! dataset.yaml
1  train: ../train/images
2  val: ../valid/images
3  test: ../test/images
4
5  nc: 3
6  names: ['Bus', 'Car', 'Truck']
7
8  roboflow:
9      workspace: image-processing-ooafx
10     project: cardetection-bcyur
11     version: 1
12     license: CC BY 4.0
13     url: https://universe.roboflow.com/image-processing-ooafx/cardetection-bcyur/dataset/1
```

2. Model Training

The proposed system was implemented in Python using the Ultralytics YOLOv8 framework.

```
C: > Users > Mina > Desktop > traffic-monitoring-system-yolov8 > traffic-monitoring-system-yolov8 > Detection > Train.py > ...
1  import torch
2  from ultralytics import YOLO
3
4  device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
5
6  # Load YOLOv8 model (e.g., yolov8n.pt for a small model)
7  model = YOLO('yolov8n.pt').to(device)
8
9  # Train the model
10 model.train(
11     data=r"C:\Users\Mina\Desktop\traffic-monitoring-system-yolov8\traffic-monitoring-system-yolov8\Detection\dataset\dataset.yaml",
12     epochs=2,
13     imgsz=640,
14     batch=16,
15     device=device
16 )
```

A pre-trained YOLOv8 Nano model (yolov8n.pt) was downloaded and used as the base model. Transfer learning was applied to adapt the pre-trained model to the custom vehicle detection dataset. The training process was then performed through the Train.py file, where the dataset configuration file was provided to the model and training parameters such as image size, batch size, and the number of epochs were specified.

Because the training was performed on a CPU-based system, the number of epochs was reduced to 2 in order to decrease the execution time.

During training, YOLOv8 loaded the pre-trained weights, scanned the training and validation datasets, automatically selected the optimizer, and displayed the learning progress in the terminal. Training statistics such as classification loss, bounding-box loss, and distribution focal loss were continuously monitored.

A sample of the training output is shown below:

```
Validating C:\Users\Wina\runs\detect\train-8\weights\best.pt...
Ultralytics 8.4.174 Python-3.12.9 torch-2.14.1+cpu CPU (11th Gen Intel Core i7-1165G7 @ 2.80GHz)
Model summary (fused): 72 layers, 3,006,233 parameters, 0 gradients, 8.1 GFLOPs

```

Class	Images	Instances	Box(P	R	mAP50	mAP50-95)
all	24	286	0.92	0.411	0.458	0.338
Bus	7	8	1	0	0.00171	0.00103
Car	24	249	0.76	0.671	0.748	0.509
Truck	15	29	1	0.562	0.623	0.504

```
Speed: 2.7ms preprocess, 137.0ms inference, 0.0ms loss, 2.9ms postprocess per image
Results saved to C:\Users\Wina\runs\detect\train-8
```

After training was completed, the generated weight files were automatically stored in:

runs/detect/train/weights/

The best.pt file was selected as the final model because it achieved the best validation performance during training.

3. Vehicle Detection and Counting

After training, the best.pt model was loaded in the Detect.py script. The input video (CarVideo.mp4) was processed frame by frame, and each frame was resized before object detection. The system detected vehicles, assigned class labels, drew bounding boxes around the detected objects, and counted the detections for each class. The processed frames were then saved to an output video file (output.avi).

The following section presents a detailed explanation of the implementation and the main components of the Detect.py code.

```
1 import cv2
2 from ultralytics import YOLO
3 import torch
4
5 # Check if CUDA is available and set the device accordingly
6 device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
7 print(f'Using device: {device}')
```

The required libraries are imported at the beginning of the program. OpenCV (cv2) is used for video processing, PyTorch (torch) is used to select the computing device, and the Ultralytics YOLO library is used for object detection. The program automatically selects the GPU if CUDA is available; otherwise, it uses the CPU. The selected device is displayed in the terminal.

```

9   # Load the trained YOLOv8 model
10  model = YOLO('best.pt').to(device)

```

The trained YOLOv8 model is loaded from the best.pt weights file. This file contains the learned model parameters obtained during training. The model is assigned to the selected computing device and is then ready to perform vehicle detection on the input video.

```

12  # Function to upscale video frame resolution
13  def upscale_frame(frame, scale_factor=2):
14      width = int(frame.shape[1] * scale_factor)
15      height = int(frame.shape[0] * scale_factor)
16      return cv2.resize(frame, (width, height), interpolation=cv2.INTER_CUBIC)

```

The upscale_frame() function increases the dimensions of a video frame using a default scale factor of 2. The new width and height are calculated by multiplying the original dimensions by the scale factor. OpenCV's bicubic interpolation method (INTER_CUBIC) is then used to resize the frame. This step increases the frame's pixel dimensions before object detection, although it does not necessarily recover additional details from the original image.

```

18  # Open video file
19  video_path = 'CarVideo.mp4'
20  cap = cv2.VideoCapture(video_path)
21
22  # Define the codec and create VideoWriter object
23  fourcc = cv2.VideoWriter_fourcc(*'XVID')
24  out = cv2.VideoWriter('output.avi', fourcc, 20.0, (int(cap.get(3) * 2), int(cap.get(4) * 2)))

```

The input video, CarVideo.mp4, is opened using OpenCV's VideoCapture function. A VideoWriter object is created to save the processed frames in output.avi. The XVID codec is used for encoding, and the output frame rate is set to 20 frames per second. The output dimensions are set to twice the input video's width and height to match the frame upscaling factor.

```

26  # Dictionary to store counts for each label
27  label_counts = {}

```

A dictionary named label_counts is initialized to store the number of detections for each vehicle class. The class name serves as the dictionary key, while its value represents the number of detections recorded for that class during video processing.

```

29  while cap.isOpened():
30      ret, frame = cap.read()
31      if not ret:
32          break
33
34      # Upscale the frame
35      frame = upscale_frame(frame, scale_factor=2)
36
37      # Perform car detection
38      results = model(frame)

```

The program processes the video frame by frame while the video capture object remains open. The `cap.read()` function reads the next frame. If no frame is returned, the loop terminates. Each successfully read frame is then enlarged by a factor of two using the `upscale_frame()` function. The resized frame is passed to the YOLOv8 model, which performs object detection and returns the results.

```
40     # Process detections for each frame
41     for result in results:
42         boxes = result.boxes # Extract the boxes object which contains all the detections
43         for box in boxes:
44             label = int(box.cls) # Class ID of the detection
45             label_name = model.names[label] # Convert class ID to class name
46             if label_name not in label_counts:
47                 label_counts[label_name] = 0
48                 label_counts[label_name] += 1
```

The detection results are examined to extract the bounding boxes of the detected objects. For each detection, the class identifier is retrieved and converted into its corresponding class name using `model.names`. If the class has not previously been recorded, a new entry is added to the `label_counts` dictionary. The counter is then incremented for every detection of that class.

This method counts detection instances across video frames rather than unique vehicles. As a result, the same vehicle may be counted multiple times if it appears in several frames.

```
50     # Render the results on the frame if detections are present
51     annotated_frame = result.plot() # Correct method to plot the results on the frame
52
53     # Display the frame (optional)
54     cv2.imshow('Object Detection', annotated_frame)
55     if cv2.waitKey(1) & 0xFF == ord('q'):
56         break
57
58     # Write the frame into the output video file
59     out.write(annotated_frame)
```

The `plot()` method draws bounding boxes and class labels on the detected objects. The resulting annotated frame is displayed in a window named Object Detection. The user can terminate the processing loop by pressing the q key. Otherwise, the annotated frame is written to the output video using the `VideoWriter` object.

```
61 cap.release()
62 out.release()
63 cv2.destroyAllWindows()
64
65 # Print the total counts for each label
66 print("Label Counts:")
67 for label, count in label_counts.items():
68     print(f"{label}: {count}")
69
70 # Calculate and print the total number of objects detected
71 total_objects = sum(label_counts.values())
72 print(f"Total Objects Detected: {total_objects}")
```

After video processing is completed or interrupted, the video capture and writer objects are released, and all OpenCV windows are closed. The program then displays the number of detections recorded for each class in the terminal. Finally, the values in the label_counts dictionary are summed to calculate and display the total number of detection instances.

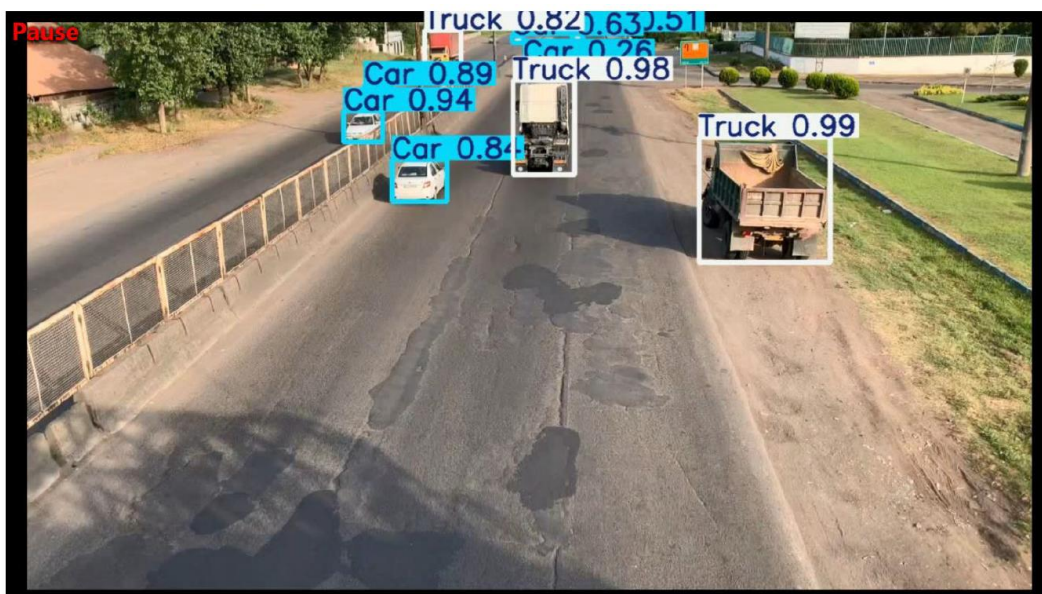
4. Detection Results

The developed system processes the traffic video frame by frame and identifies vehicles according to the knowledge learned during training.

The following figure illustrates an example frame before processing.



After inference, the detected vehicles are highlighted using bounding boxes and class labels.



At the end of the execution, the software displays the number of detected objects for each class and the total number of detected vehicles in the terminal.

```
0: 640x640 1 Bus, 11 Cars, 2 Trucks, 183.5ms
Speed: 4.0ms preprocess, 183.5ms inference, 1.0ms postprocess per image at shape (1, 3, 640, 640)

0: 640x640 1 Bus, 11 Cars, 1 Truck, 185.5ms
Speed: 5.0ms preprocess, 185.5ms inference, 0.0ms postprocess per image at shape (1, 3, 640, 640)

0: 640x640 1 Bus, 11 Cars, 1 Truck, 175.5ms
Speed: 6.0ms preprocess, 175.5ms inference, 1.0ms postprocess per image at shape (1, 3, 640, 640)

0: 640x640 1 Bus, 12 Cars, 1 Truck, 179.5ms
Speed: 4.0ms preprocess, 179.5ms inference, 1.0ms postprocess per image at shape (1, 3, 640, 640)

0: 640x640 12 Cars, 1 Truck, 169.5ms
Speed: 4.0ms preprocess, 169.5ms inference, 0.0ms postprocess per image at shape (1, 3, 640, 640)

0: 640x640 11 Cars, 1 Truck, 175.5ms
Speed: 5.0ms preprocess, 175.5ms inference, 1.0ms postprocess per image at shape (1, 3, 640, 640)

0: 640x640 12 Cars, 1 Truck, 176.5ms
Speed: 4.0ms preprocess, 176.5ms inference, 1.0ms postprocess per image at shape (1, 3, 640, 640)

0: 640x640 12 Cars, 1 Truck, 170.5ms
Speed: 4.0ms preprocess, 170.5ms inference, 1.0ms postprocess per image at shape (1, 3, 640, 640)

0: 640x640 12 Cars, 1 Truck, 177.5ms
```

The results demonstrate that the proposed system can successfully detect, classify, label, and count vehicles in traffic videos using a custom-trained YOLOv8 model.